MODULE 24 · DOMAIN 4 - ASSEMBLING AND DEPLOYING APPLICATIONS

Serving: Foundation Model APIs, throughput and ai_query

Five ways to run a model, how to choose between them, and batch inference over whole tables.

SECTION 4 of 6 EXAM WEIGHT 22% MODULE 24 of 40

admin@sthithapragna.com

A taxi, a priority taxi, your own car, or a freight lorry. **Pay per trip, pay extra to jump the queue, own the capacity, or move everything at once.**

Five ways to run a model on Databricks

MODE	DATABRICKS RECOMMENDS IT FOR
Pay-per-token	Getting started. Shared pool, best effort
Priority pay-per-token	Latency-sensitive production, no commitment, higher rate
Provisioned throughput	Production needing guarantees, fine-tuned models, compliance
External models	Models hosted outside Databricks, governed centrally
AI Functions batch	Batch inference, with batch-optimised models

ON THE EXAM

The recommendations come straight from Databricks' own description of each mode. Learn the phrase that goes with each.

Pay-per-token, or provisioned throughput?

PAY-PER-TOKEN

Pay as you go

- Already available in the workspace
- Shared pool, best effort
- Right for exploring and spiky use
- No fine-tuned models

PROVISIONED THROUGHPUT

Dedicated capacity

- Performance guarantees
- Fine-tuned and custom weights
- Compliance certifications, such as HIPAA
- Recommended for production

THE TIEBREAKER

Prototypes and unpredictable internal use: pay-per-token. Guarantees, fine-tuned weights or compliance: provisioned throughput.

Priority pay-per-token, or provisioned throughput?

PRIORITY PAY-PER-TOKEN

Jump the queue

- service_tier set to priority, per request
- Admitted ahead of standard traffic
- No commitment, higher token rate
- More consistent latency under load

PROVISIONED THROUGHPUT

Own the capacity

- On-demand, or reserved for 1 or 3 months
- Dedicated, guaranteed capacity
- Reserved overflow spills to priority
- Required for fine-tuned weights

THE TIEBREAKER

Consistent latency without a commitment: priority. Guaranteed capacity, predictable traffic or fine-tuned weights: provisioned throughput.

Models hosted somewhere else

- An endpoint on Databricks that **forwards to an outside provider**.
- The provider's API key lives in **Databricks secrets**, not in code.
- Governed like any endpoint: permissions, rate limits, usage tracking, logging.
- **Fallbacks** to another model work for external model endpoints.
- The model still runs at the provider. Its terms and data handling still apply.

> ON THE EXAM
>
> When a company already uses an outside provider, an external model endpoint brings it under Databricks governance without moving the model.

An interactive endpoint, or ai_query batch?

INTERACTIVE ENDPOINT

Someone is waiting

- One request at a time
- Latency per request matters
- Chat, apps, agents
- OpenAI-compatible clients

AI_QUERY BATCH

A table to process

- Thousands to millions of rows
- Throughput matters
- SQL or DataFrame, results to a table
- Parallelism and retries handled for you

THE TIEBREAKER

From module 02: if nobody is waiting per row, it is batch. Batch belongs in ai_query, not in a loop calling an endpoint.

The arguments

ARGUMENT	PURPOSE
endpoint	The model endpoint or model service to call
request	The prompt, often built with CONCAT from columns
returnType	The output type, for custom model endpoints
failOnError	Default true. False returns response and error per row
modelParameters	Constant settings, such as max_tokens. Temperature defaults to 0
responseFormat	A JSON schema, for structured output into typed columns

ON THE EXAM

failOnError false and responseFormat are the two arguments exam scenarios most often turn on.

Databricks' own advice for ai_query at scale

- Use a **batch-optimised model**. Others run at reduced throughput.
- **Submit the whole dataset in one query.** AI Functions parallelise and retry for you.
- Splitting data into small batches yourself **reduces** throughput.
- Set **failOnError to false** for large jobs, then review failed rows.
- Queries to system.ai model services route through Unity AI Gateway.

> ON THE EXAM
>
> The instinct to chunk a big job into small batches is wrong here. One query over the whole table is faster.

Pennant Outdoor matches each job to a mode

JOB	MODE
Trying three models for a new feature	Pay-per-token
Customer chat, spiky, must feel fast	Priority pay-per-token
Fine-tuned product-tagging model	Provisioned throughput
Existing contract with an outside provider	External model endpoint
Nightly summaries of 2 million reviews	ai_query, one query, failOnError false

ON THE EXAM

Five jobs, five modes. The deciding detail was different each time: stage, latency, weights, contract, volume.

> A team is spending one week comparing three hosted models for a proof of concept.
>
> Traffic is light and unpredictable, and nobody wants a long-term commitment yet.

Which way of running the models fits?

A Reserve provisioned throughput for a three-month term

B Pay-per-token endpoints already available in the workspace

C Fine-tune each model and serve it on provisioned throughput

D Send every test prompt through a nightly ai_query job instead

◆ PAUSE AND DECIDE

Which way of running the models fits?

A Reserved for 3 months

A term commitment for a one-week experiment.

B Pay-per-token

Ready now, pay per use, recommended for getting started.

C Fine-tune, then provision

Weeks of work before the comparison even starts.

D Nightly ai_query

Interactive testing becomes a day-long feedback loop.

THE SIGNAL

One week, light traffic, no commitment. Databricks recommends pay-per-token for getting started.

> A team has fine-tuned an open Llama model on its own support transcripts and registered the weights in Unity Catalog.
>
> It now needs to serve the fine-tuned model to an application.

How should the team serve it?

A. Use the pay-per-token endpoint for the base model instead

B. Call priority pay-per-token with the service_tier flag set

C. Create an external model endpoint pointing at the weights

D. Create a provisioned throughput endpoint for the weights

◆ PAUSE AND DECIDE

How should the team serve it?

A Base model endpoint

Serves the original model, not the fine-tuned one.

B Priority pay-per-token

Still the shared, hosted models. Not your weights.

C External model endpoint

For models hosted outside Databricks. These are inside.

D Provisioned throughput

The mode that serves fine-tuned and custom weights.

THE SIGNAL

Fine-tuned weights in Unity Catalog. Only provisioned throughput serves your own weights.

> A customer chat assistant is launching. Traffic will be spiky and hard to predict, and replies must stay fast even at busy times.
>
> Finance will not approve any term commitment until usage is known.

Which mode fits best?

A. Priority pay-per-token, with service_tier set to priority

B. Standard pay-per-token, with automatic retries on any error

C. Reserved provisioned throughput for a fixed three-month term

D. A scheduled ai_query job, with cached answers for customers

◆ PAUSE AND DECIDE

Which mode fits best?

A Priority pay-per-token

Consistent latency under load, with no commitment.

B Standard, with retries

Best effort. Retries add delay exactly when busy.

C Reserved for 3 months

A term commitment finance has refused.

D Scheduled job, cached

Customers ask new questions. Caching cannot answer them.

THE SIGNAL

Fast under load, spiky traffic, and no commitment. That combination is exactly what priority mode is for.

admin@sthithapragna.com

> A hospital group's clinical documentation assistant is business-critical, with steady, predictable daytime traffic.
>
> It needs guaranteed capacity and an endpoint covered by HIPAA compliance certification.

Which mode fits best?

A Standard pay-per-token, on the shared best-effort pool

B Priority pay-per-token, set on every single request

C Provisioned throughput, reserved for a fixed term

D An external model endpoint with a fallback configured

◆ PAUSE AND DECIDE

Which mode fits best?

A **Standard pay-per-token**

Best effort is the opposite of guaranteed.

B **Priority pay-per-token**

Better queue position, still not guaranteed capacity.

C **Reserved provisioned throughput**

Guaranteed capacity, compliance certifications, steady traffic.

D **External with fallback**

Moves the model outside, adding a provider to the compliance scope.

THE SIGNAL

Guaranteed capacity, HIPAA, business-critical and predictable. Every phrase is in Databricks' description of provisioned throughput.

> Every night a team summarises 10 million support tickets with an LLM, writing results to a table.
>
> Its script splits the table into 1,000 small batches and loops over them. The job is slow.

What does Databricks recommend?

A Call the chat endpoint row by row from a Python loop

B Split the data into even smaller batches of 100 rows

C Move the loop to priority pay-per-token so it goes faster

D Submit the whole table in one ai_query over a batch model

◆ PAUSE AND DECIDE

What does Databricks recommend?

A Row by row

The slowest possible shape for a batch job.

B Smaller batches

Manual splitting reduces throughput. Smaller is worse.

C Priority mode

Built for interactive latency, at a higher rate, not batch.

D One ai_query, batch model

AI Functions parallelise and retry for you.

THE SIGNAL

Splitting into small batches is the instinct Databricks' guidance corrects. Submit the full dataset in one query.

> A nightly ai_query over 5 million rows fails at row 3.9 million, because the endpoint rejects a handful of malformed rows.
>
> The whole job then reruns from the start, and the deadline is missed.

What should the engineer change?

A Set failOnError to false, then review the failed rows

B Wrap the query in a loop that restarts it after failure

C Lower the temperature so fewer rows get rejected

D Split the job into smaller batches of one thousand

◆ PAUSE AND DECIDE

What should the engineer change?

A failOnError false

The job completes; failed rows carry an error to review.

B Restart loop

Reruns everything, and fails again on the same rows.

C Lower temperature

Malformed input is rejected whatever the temperature.

D Smaller batches

Reduces throughput, and still needs error handling.

THE SIGNAL

A few bad rows sink five million good ones. failOnError false keeps the successes and flags the failures.

A batch job uses ai_query to pull the product, fault and severity from each warranty email into separate columns.

The model's text output varies in shape, so parsing it into columns keeps breaking.

Which change makes the output reliably typed?

A Parse the text afterwards with regular expressions

B Raise max_tokens in modelParameters so JSON completes

C Pass a JSON schema through the responseFormat argument

D Set failOnError to false so malformed rows are dropped

◆ PAUSE AND DECIDE

Which change makes the output reliably typed?

A Regular expressions

Chases the varying shape instead of fixing it.

B More max_tokens

Nothing says outputs are cut off. Shape is the problem.

C responseFormat schema

Constrains output to the schema, parsed into typed columns.

D failOnError false

Keeps the job running, but does not fix the shape.

THE SIGNAL

Varying shape, and typed columns wanted. responseFormat is structured output for batch, just as module 03 was for chat.

> A company already pays for a model from an outside provider. It wants Databricks apps to use that model.
>
> Security requires central governance and usage tracking, and no API keys in code.

What should the engineer set up?

A Call the provider directly, with the key stored in the notebook

B An external model endpoint, with the key held in Databricks secrets

C Download the provider's model and register it in Unity Catalog

D Ask the provider to host the model on a pay-per-token endpoint

◆ PAUSE AND DECIDE

What should the engineer set up?

A Direct call, key in notebook

A key in code, and no central governance.

B External model endpoint

Governed, tracked, and the key stays in secrets.

C Download and register

Proprietary provider models are not downloadable.

D Provider on pay-per-token

Not something a customer can arrange.

THE SIGNAL

An outside provider, central governance, no keys in code. External model endpoints exist for exactly this.

What to carry forward

- **Pay-per-token** to start. **Priority** for fast, spiky, uncommitted production.
- **Provisioned throughput**: guarantees, fine-tuned weights, compliance. On-demand or reserved.
- **External models**: outside providers, governed, keys in secrets.
- Batch: **one ai_query over the whole table**, batch-optimised model, failOnError false.
- responseFormat for typed columns. Temperature defaults to 0 in ai_query.

> ON THE EXAM
>
> Next: module 25 controls access from serving endpoints: whose identity, which secrets, which permissions.